Export the daily plan as an ICS file

Add export options for calendar sync
Optimize study block distribution
Style the heatmap with Tailwind CSS
Add unit tests for workload calculations

Built the calendar-export prototype, improved study-block distribution, added Tailwind heatmap styling, and created runnable unit tests. The package includes a sample ICS file with 22 fictional study blocks totaling 660 minutes for October 7–13, 2026—not a copy of your personal calendar.

All 22 unit tests passed. No connected calendar was changed, and live calendar sync remains separate from file export.

1. Calendar export options

The dashboard now exports confirmed blocks—not unconfirmed previews.

Option	Implemented behavior
ICS snapshot	Download selected study blocks for calendar import
Private-plan JSON	Export block identifiers, times, durations, and timezone
Scope filter	Next seven days or all confirmed blocks
Course filter	Export one course or all courses
Title privacy	Generic “Private study block” by default; course/assignment titles optional
Reminder	None, 10 minutes, or 30 minutes before
Live sync	Not implemented
Calendar subscription	Not implemented; requires a hosted feed

The ICS serializer uses UTC event times, persistent block UIDs, CRLF line endings, escaped text, and UTF-8-aware line folding. These follow iCalendar’s exchange-format requirements.[1]

Export call

The reusable module exposes:

```
const calendarText = createICS(confirmedBlocks, {
  tasks,
  includeDetails: false,
```

```
  reminderMinutes: 10,
  generatedAt: Date.now()
});
```

Each confirmed block needs a persistent ID:

```
{
  id: crypto.randomUUID(),
  taskId: "bio",
  start: "2026-10-08T18:00",
  minutes: 30
}
```

The dashboard assigns the ID when the block is confirmed and preserves it in local storage. Repeated exports retain that UID while the same block remains stored.

Import is not sync

An ICS import is a snapshot. Subsequent edits in Semester do not automatically update or delete imported events. Outlook explicitly distinguishes file import from subscribing to an online calendar.[2]

Stable UIDs also do not guarantee that every client will deduplicate repeated imports. Use a separate study calendar while testing.

For future live sync, the activation requirements are:

- A resolved destination calendar and authorized connector.
- User approval before calendar writes.
- A mapping between Semester block IDs and provider event IDs.
- Idempotent creates, updates, and deletions.
- Conflict handling, reconciliation, and revocation.

No OAuth tokens, official grades, or grade-target scenarios are included in the exports. `CLASS:PRIVATE` is metadata—not encryption.

2. Improved block distribution

The previous planner filled early feasible slots first. The new planner spreads work by selecting the least-loaded feasible day.

Selection sequence

```
Rank eligible assignments
        ↓
Subtract already-planned future minutes
        ↓
Find slots satisfying deadline and availability
        ↓
```

```
Exclude conflicts and enforce breaks
        ↓
Enforce daily workload and course caps
        ↓
Choose lowest daily load
        ↓
Tie-break by course load, then earliest start
        ↓
Report anything that cannot fit
```

Existing blocks remain fixed. The generator proposes additions; it does not silently move commitments.

Configurable constraints

Setting	Default
Planning horizon	Seven days
Study start	18:00
Daily availability window	180 minutes
Break between blocks	10 minutes
Maximum study per course per day	90 minutes
Normal session	30 minutes
Final smaller session	15 minutes

Breaks consume availability. A three-hour window therefore does not necessarily accommodate three hours of study.

Verified distribution example

For 180 minutes of work with three feasible days:

Day	Generated study time
Day 1	60 minutes
Day 2	60 minutes
Day 3	60 minutes

Other executed tests confirmed that the planner:

- Maintains 10-minute spacing between generated 30-minute sessions.
- Reports 120 minutes unscheduled when a one-day course cap permits only 60 of 180 requested minutes.
- Never generates a start earlier than the supplied current time.

This is a balancing heuristic, not a globally optimal solver or a guarantee of learning outcomes.

3. Tailwind heatmap styling

The updated HTML includes Tailwind utility classes and an offline CSS fallback.

Responsive container

```
<div
  id="heat"
  class="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-7"
  aria-label="Seven-day workload"
></div>
```

Explicit status classes

Use complete class names rather than assembling partial Tailwind names:

```
const tones = {
  Empty: "bg-white border-slate-300",
  Light: "bg-slate-100 border-slate-400",
  Balanced: "bg-blue-100 border-blue-400",
  "Over capacity": "bg-rose-100 border-rose-500"
};
```

Day card

```
<article
  class="rounded-md border border-blue-400 bg-blue-100
         p-4 text-slate-900"
>
  <h3 class="font-semibold tabular-nums">
    2026-10-08
  </h3>

  <p class="text-2xl font-semibold tabular-nums">
    120 min
  </p>

  <p class="text-sm font-medium">
    Balanced
  </p>

  <p class="mt-2 text-sm">
    1 due · 180 estimated min due
  </p>
</article>
```

Color is accompanied by the workload label and minute totals.

The prototype uses Tailwind's Play CDN for development. Tailwind explicitly says that this CDN is not intended for production; compile static CSS through Semester's existing build tooling

before deployment.[3]

Because the development script is externally loaded and can access same-origin data, this prototype should contain only fictional or appropriate non-sensitive data.

4. Workload unit tests

The attached test suite uses Node's built-in test runner.

Run it alongside the reusable module:

```
TZ=America/Chicago node --test semester-workload-calendar.test.cjs
```

PowerShell:

```
$env:TZ = "America/Chicago"
node --test semester-workload-calendar.test.cjs
```

Executed coverage

Group	Tests	Coverage
Workload calculations	12	Empty days, exact thresholds, midnight splitting, due effort, exclusions, invalid inputs, DST elapsed time
Planner	4	Balanced distribution, breaks, course caps, past-start exclusion
ICS serializer	6	UTC/CRLF, escaping, Unicode folding, stable UIDs, reminders, duplicate-ID rejection
Total	22	All passed

Example midnight test

```
test("cross midnight split", () => {
  const result = workloadDays({
    blocks: [
      {
        start: "2026-10-07T23:30",
        minutes: 90
      }
    ],
    tasks: [],
    effort: {},
    now: parseLocalDateTime("2026-10-07T12:00"),
    days: 2,
    capacity: 120
  });

  assert.equal(result[^0].plannedMinutes, 30);
  assert.equal(result[^1].plannedMinutes, 60);
});
```

The heatmap counts actual interval overlap with each day—not the entire block on its starting date.

Example capacity test

```
test("balanced at exactly 100%", () => {
  const result = workloadDays({
    blocks: [
      {
        start: "2026-10-07T18:00",
        minutes: 120
      }
    ],
    tasks: [],
    effort: {},
    now: parseLocalDateTime("2026-10-07T12:00"),
    days: 1,
    capacity: 120
  });

  assert.equal(result[^0].level, "Balanced");
});
```

Calendar-client import interoperability, browser interaction, accessibility, and compiled Tailwind styling have not yet been tested.

5. Use the updated prototype

The delivered package contains:

- Complete calendar-export and Tailwind dashboard HTML.
- Reusable calendar/planning module.
- Runnable workload, planner, and ICS tests.
- Fictional demonstration ICS file.
- Integration guide.

The local workflow is:

1. Edit remaining-effort estimates.
2. Set availability, breaks, and the daily course cap.
3. Preview the balanced plan.
4. Review unscheduled work.
5. Confirm private study blocks.
6. Inspect the updated heatmap.
7. Choose export scope, title detail, and reminder.
8. Download the ICS snapshot or private-plan JSON.

Serve the local folder with:

```
python -m http.server 8000
```

Open the dashboard through localhost. The demonstration ICS is already generated; exporting your actual browser plan requires confirming blocks in that dashboard first.

The local features are implemented and unit-tested. Production integration, institutional authorization, subscription feeds, and live calendar writes remain separate activation gates.

⁂

1. https://datatracker.ietf.org/doc/html/rfc5545
2. https://support.microsoft.com/en-us/outlook/import-or-subscribe-to-a-calendar-in-outlook-com-or-outlook-on-the-web
3. https://tailwindcss.com/docs/installation/play-cdn
4. https://www.ablebits.com/office-addins-blog/add-google-calendar-outlook/
5. https://dev.to/sendotltd/building-an-rfc-5545-ical-file-generator-line-folding-escaping-and-all-5fid
6. https://github.com/ical-org/ical.net/wiki/iCalendar-Specification-(RFC5545)
7. https://www.usecarly.com/blog/how-to-add-google-calendar-to-outlook/
8. https://www.hetk.io/blog/how-to-subscribe-ics-calendar-outlook/
9. https://itsourcecode.com/blogs/how-to-sync-google-calendar-with-outlook-2026/
10. https://support.microsoft.com/en-us/outlook/calendar/see-your-google-calendar-in-outlook
11. https://www.reddit.com/r/tailwindcss/comments/1bpmptc/what_if_i_just_use_playcdn_in_production/
12. https://dev.to/sendotltd/rfc-5545-is-a-200-page-spec-your-icalendar-feed-needs-about-150-lines-51k7
13. https://tailkits.com/blog/tailwind-css-v4-cdn-setup/
14. https://github.com/tailwindlabs/tailwindcss/issues/10369
15. https://www.usecarly.com/blog/how-to-import-calendar-into-outlook/